





**Ciência da Computação**

Estrutura de Dados

# Listas

Aldo Moura
aldo.moura@souunit.com.br

# Objetivo e Roteiro da Aula



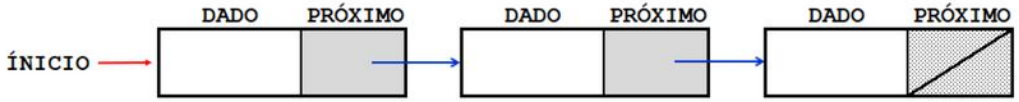
Conhecer o conceito de lista simplesmente encadeada e manipular sua células.

- Lista simplesmente encadeada
  - Inserir célula
  - Imprimir células
  - Buscar célula
  - Remover célula
- Outros tipos de listas

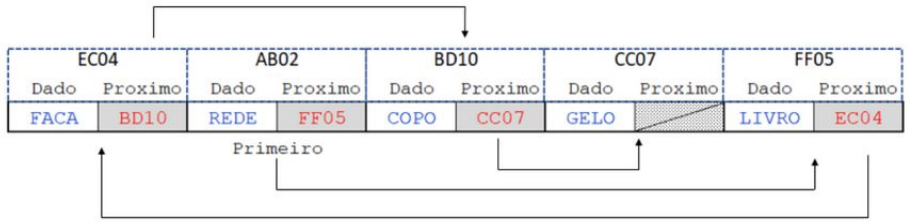
# Lista Simplesmente Encadeada

**Também denominada apenas de lista encadeada (Lista ligada ou *linked list*)**

- É uma **representação** de uma sequência de elementos, todos do mesmo tipo, na memória RAM do computador.
- Sequência dos elementos - Cada elemento é armazenado em uma célula da lista: o primeiro elemento na primeira célula, o segundo na segunda célula, e assim por diante.
- Cada célula contém um dado e o endereço da célula seguinte.



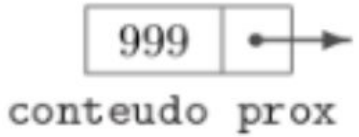

- A representação gráfica causa impressão de que as células ocupam posições consecutivas na memória. No entanto as células podem estar alocadas de maneira descontínua na memória.



# Lista Simplesmente Encadeada

Definindo a estrutura da célula para uma lista:

```
typedef struct reg
{
    int conteudo;
    struct reg *prox;
} celula;
```

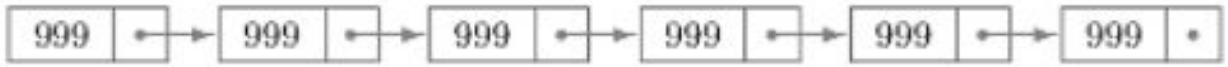



Declarando uma variável para armazenar um célula da lista:

```
celula c;
```

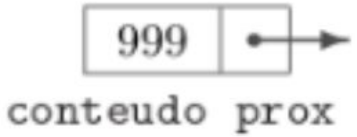
Declarando um ponteiro  para apontar para célula do mesmo tipo da célula declarada:

```
celula *p;
```



# Lista Simplesmente Encadeada

- Sendo c uma célula:
  - **c.conteudo** é o conteúdo da célula
  - **c.prox** é o endereço da próxima célula

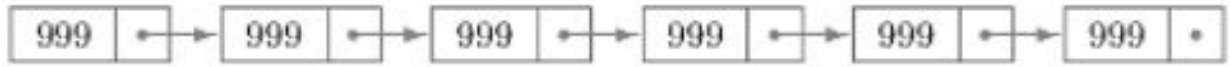



- Sendo p o endereço de uma célula
  - **p->conteudo** é o conteúdo da célula apontada
  - **p->prox** é o endereço da próxima célula

```
typedef struct reg
{
   int conteudo;
   struct reg *prox;
} celula;

celula c;
celula *p;
```

- Sendo p o endereço da *última* célula da lista
  - **p->prox** vale **NULL**



# Lista Simplesmente Encadeada

- O endereço de uma lista encadeada é o endereço de sua primeira célula.
- Se **le** é o endereço de uma lista encadeada, dizemos simplesmente que

  ***le*** *é uma lista encadeada.*

- A lista está vazia se e somente se le == NULL.

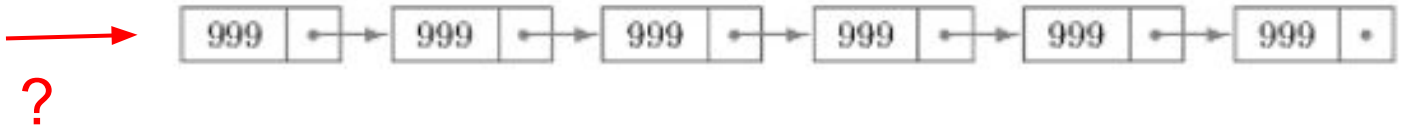
- Listas são estruturas eminentemente recursivos.

  Se **le** é uma lista não vazia então le->prox também é uma lista.

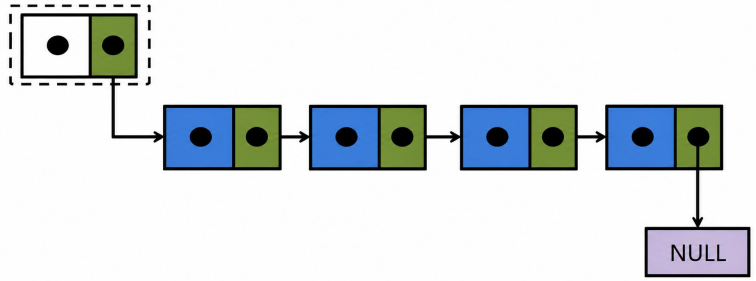
  Muitos algoritmos sobre listas encadeadas ficam mais simples quando escritos em estilo recursivo.

# Lista Encadeada Com Cabeça

Como cada célula aponta para a próxima célula (exceto a última), nenhuma célula aponta para a primeira célula, tornando impossível a inserção de nova célula no início da lista




- Alternativa: implementar uma lista tratando a primeira célula apenas como um marcador de início da lista e ignorando o conteúdo da célula.
- A primeira célula é denominada de **célula cabeça** (*head cell*) e a lista denominada **lista com cabeça**.
- Se a lista tem cabeça, para inserir no início da lista  basta p aponta para a célula-cabeça.



# Cabeça de Lista Encadeada

- Uma lista encadeada le com cabeça está vazia se e somente se le->prox == NULL.
- Criando uma lista encadeada vazia com cabeça, basta dizer:

```
celula *le;
le = (celula*) malloc( sizeof (celula) );
le -> prox = NULL;
```

Alocação dinâmica de memória:
malloc: Faz a alocação dinâmica de memória.

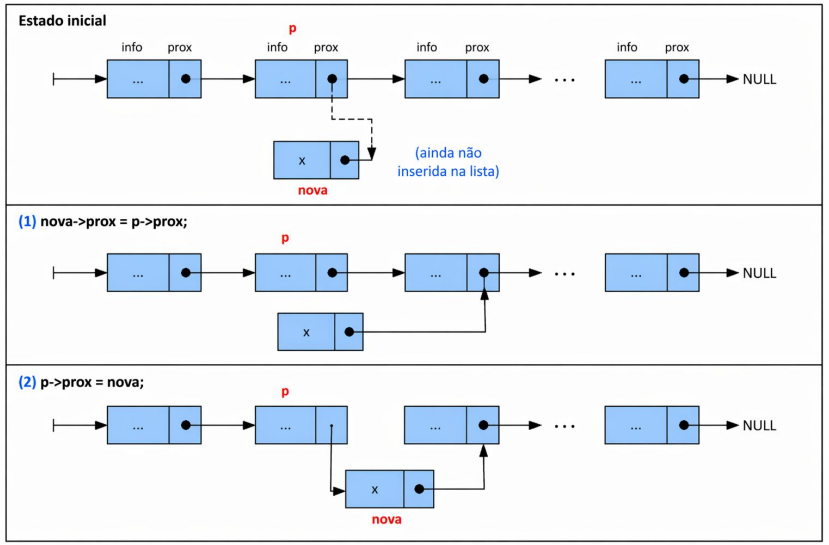
- Recebe como argumento um inteiro sem sinal, representando a quantidade em bytes de memória requerida.
- Retorna um ponteiro para o primeiro byte de memória que foi alocado.

# Inserir em Lista Encadeada

Insere uma nova célula em uma lista encadeada entre a célula p e a célula seguinte.
x = conteúdo que será armazenado
p = endereço da célula depois da qual a nova célula será inserida. Supõe-se p seja diferente NULL.

- O campo conteúdo da nova célula recebe o valor de x
- (1) A nova célula passa a apontar para a célula seguinte, ao receber em seu campo proximo o endereço da célula seguinte que ainda está sendo apontado pelo campo próximo da célula p.
- (2) A célula p passa a apontar para o endereço da nova célula ao receber em seu campo proximo o endereço da nova célula

```
void inserir(int x, celula *p)
{
    celula *nova;
    nova=(celula*) malloc (sizeof (celula));
    nova->conteudo = x;
    nova->prox = p->prox;     // (1)
    p->prox = nova;           // (2)
}
```



# Imprimir Lista Encadeada Com Cabeça

```
// Versão iterativa


void imprimir (celula *le)
{
    celula *p;
    for (p = le->prox; p != NULL; p = p->prox)
    {
        printf ("%d\t", p->conteudo);
    }
   printf ("\n");
}
```

```
// Versão recursiva

void imprimir(celula *le)
{
    if (le->prox == NULL)
    {
        printf("\n");
        return;
    }

    printf("%d\t", le->prox->conteudo);

    imprimir(le->prox);
}
```

# Busca em Lista Encadeada (Versão Iterativa)

Verificar se um objeto x pertence a uma lista encadeada, ou seja, se é igual ao conteúdo de alguma célula da lista:

```
/* Esta função recebe um inteiro x e uma lista encadeada le
   e retorna o endereço da célula que contém x.
   Se o inteiro não for encontrado, retorna NULL. */
celula *buscar(int x, celula *le)
{
    celula *p;
    p = le;
    while (p != NULL && p->conteudo != x)
    {
        p = p->prox;
    }
    return p;
}
```

# Busca em Lista Encadeada (Versão Recursiva)

Verificar se um objeto x pertence a uma lista encadeada, ou seja, se é igual ao conteúdo de alguma célula da lista:

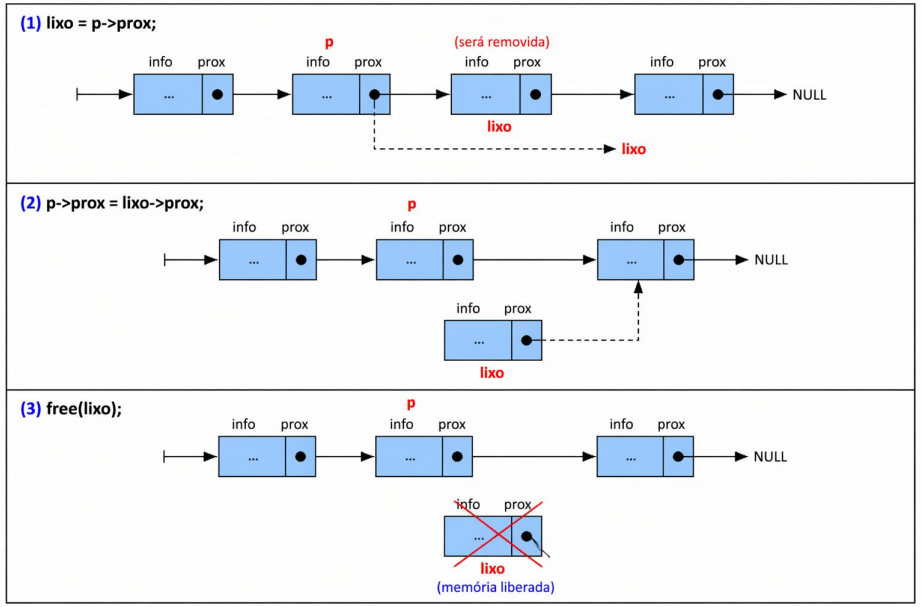
```
/* Esta função recebe um inteiro x e uma lista encadeada le
   e retorna o endereço da célula que contém x.
   Se o inteiro não for encontrado, retorna NULL. */
celula *buscar(int x, celula *le)
{
    if (le == NULL)
    {
        return NULL;
    }
    if (le->conteudo == x)
    {
        return le;
    }
    return buscar(x, le->prox);
}
```

# Remover em Lista Encadeada

Remove uma célula em uma lista encadeada.
A célula a ser removida é a célula seguinte ao endereço informado.
p = endereço da célula depois da qual a célula será removida. Supõe-se p e p->prox diferentes de NULL.

1. Um ponteiro é criado para apontar para o endereço da célula que será removida, que corresponde ao endereço apontado pelo ponteiro p (**p->proximo**).
2. O campo proximo (**p->proximo**) passa a apontar para o endereço da célula seguinte à célula que será removida.
3. A memória da célula que será removida é liberada.

```
void remover(celula *p)
{
    celula *lixo = p->prox; // (1)
    p->prox = lixo->prox;   // (2)
    free(lixo);             // (3)
}
```



# Remover em Lista Encadeada

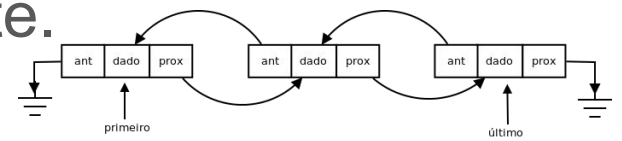
- Não é preciso copiar informações de um lugar para outro, como para remover um elemento de um vetor: basta mudar o valor de um ponteiro.  A função consome sempre o mesmo tempo, quer a célula a ser removida esteja perto do início da lista, quer esteja perto do fim.

- Note também que a função de remoção não precisa conhecer o endereço da lista, ou seja, não precisa saber onde a lista começa.

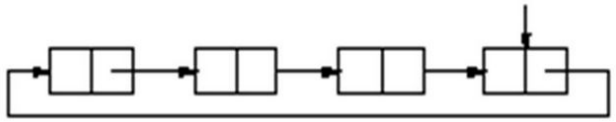
# Outros tipos de Listas

Uma vez entendidas as listas encadeadas básicas, pode-se estudar os outros tipos de listas encadeadas.

- **Lista duplamente *encadeada***: cada célula contém o endereço da célula anterior e o endereço da célula seguinte.




- **Lista encadeada *circular***: a última célula aponta para a primeira. O endereço de uma tal lista é o endereço de qualquer uma de suas células.



- **Lista duplamente encadeada *circular***: a última célula aponta para a primeira e a primeira célula aponta para a última.

